Task (Phase 0, do this FIRST): AWS render environment, using EC2 + S3 + Blender

Goal: the same container that runs locally must also run on AWS GPU instances. It renders dataset shards and writes them to S3. This must work before any scene work starts (`docs/TASK_scenes.md`), so every later phase can be tested at scale.

Follow `AGENTS.md`. Deliver it in the steps below. Each step ends with its acceptance check passing and the result reported in the PR.

Design decisions (already made, do not change them without asking)

Topic	Decision
Infrastructure as code	**Terraform** in `infra/terraform/`. Everything is tagged, and everything can be destroyed with one command. No click-ops.
Runtime	**One Docker image** (`infra/docker/Dockerfile`), used locally, on the dev box, and in batch jobs. It is pushed to **ECR**.
Blender	Pinned to **4.2 LTS**. The version, the image digest and the git commit are written into every dataset's `manifest.json` (see step 4).
Interactive work	One **GPU dev instance**, reached through **SSM Session Manager** (no SSH keys, no open inbound ports). It auto-stops when idle.
Scale-out	**AWS Batch** on EC2 GPU instances, **Spot** first with an On-Demand fallback. Jobs run as **array jobs**: array index = shard. The compute environment scales to 0 vCPU when idle.
Storage	One private **S3** bucket for datasets. The worker renders to local NVMe/EBS scratch, then uploads each finished frame directory.
Credentials	IAM roles only (an instance profile and a Batch job role, least privilege, limited to this bucket and ECR repo). No access keys anywhere in the repo, the image, or the env files.
Cloud awareness	`synthrig/` stays cloud-agnostic. All AWS code lives in `infra/`. The pipeline keeps writing to a local directory, and `infra/worker.py` wraps it and uploads.

Configuration: `infra/config.yaml` (the ONLY place for account-specific values)

```
project: synthrig
owner: <email or team>              # used in tags
cost_center: <code>                 # tag, if our org requires it
aws_profile: <sso profile name>      # local CLI profile (SSO), never keys
region: us-east-1                    # confirm with cloud team
```

```
vpc_id: null                          # null = create a minimal VPC; else use
existing (org may require this)
subnet_ids: []                        # private subnets with NAT or VPC endpoints
bucket: <globally-unique-name>
ecr_repo: synthrig
dev_instance_type: g6.xlarge          # NVIDIA L4; alternatives g5.xlarge (A10G),
g4dn.xlarge (T4)
batch_instance_types: [g6.xlarge, g6.2xlarge, g5.xlarge, g5.2xlarge]
batch_max_vcpus: 64
use_spot: true
dev_idle_stop_minutes: 60
monthly_budget_usd: <number>          # creates an AWS Budget alert to `owner`
```

Organisations often mandate an existing VPC, approved AMIs, permission boundaries or required tags. Make each of these a config value, not a code change. Do not hard-code instance prices anywhere.

Target S3 layout

```
s3://<bucket>/datasets/<scene>/<rig_hash>/<run_id>/
    manifest.json  rig.yaml  calib/...          # written once by `submit`,
before shards start
    frames/<NNNNNN>/...                          # uploaded by workers, one
frame dir at a time
    frames/<NNNNNN>/_DONE                        # written LAST for each frame
=> resumable, idempotent
    logs/shard_<k>.log
s3://<bucket>/scratch/...                        # lifecycle rule: expire after
14 days
```

`run_id` = UTC timestamp + short git sha. A frame counts as done only if it has `_DONE`. Workers skip done frames, so re-running a failed array job never duplicates work.

Step 0.1: container (local only, no AWS yet)

`infra/docker/Dockerfile`:

- Base: an NVIDIA CUDA runtime image (Ubuntu 22.04), so Cycles can use CUDA/OptiX when a GPU exists and fall back to CPU when not.
- Install Python 3.11 plus the repo's dependencies, and Blender 4.2 LTS. Two options. Implement **A** first; switch to **B** only if the GPU check in step 0.3 fails:
  - **A:** `pip install bpy==4.2.0` (the current dev path).
  - **B:** the official Blender 4.2 LTS Linux tarball (checksum verified), with the repo's deps pip-installed into Blender's bundled Python. Run via `blender -b --python run.py -- <args>`.
- Install Blender's headless runtime libraries (libGL, libX11/Xi/Xrender/Xxf86vm, libxkbcommon, libSM, etc.). Keep the image small: multi-stage build, no build toolchain in the final layer.

- Entrypoint: `infra/worker.py` (step 0.4). `docker run <image> pytest` must also work.

`infra/cloud.py` is a small Python CLI (argparse + boto3; subprocess for docker/terraform). It is the single entry point for everything in this task. Start with:

```
python infra/cloud.py image build            # docker build, tag with git sha
python infra/cloud.py image test             # docker run ... pytest -q  (+ 1-
frame CPU render at scale 0.125)
```

Accept: `image test` passes on a laptop without a GPU. The rendered frame opens with `eval/preview.py`.

Step 0.2: base infrastructure (Terraform)

`infra/terraform/` must create:

- **The S3 bucket:**
  - Block all public access.
  - SSE-S3 (or KMS if the org requires it).
  - A lifecycle rule on `scratch/`.
  - Abort incomplete multipart uploads after 7 days.
- **Networking, only if `vpc_id` is null:**
  - A minimal VPC.
  - A gateway VPC endpoint for S3, so render traffic avoids the NAT.
  - Interface endpoints for ECR, SSM and CloudWatch Logs, so private instances work without public IPs.
- **The ECR repo:** scan-on-push enabled, and a lifecycle rule that keeps the last 20 images.
- **IAM:**
  - An instance/job role with read/write on `s3://<bucket>/datasets/*` and `scratch/*` only.
  - ECR pull.
  - CloudWatch Logs.
  - `AmazonSSMManagedInstanceCore` for the dev box.
- **An AWS Budget** with an email alert to `owner`.
- **Tags on every resource:** `project`, `owner`, `cost_center`, `managed_by=terraform`.

CLI:

```
python infra/cloud.py infra plan | apply | destroy
python infra/cloud.py image push             # push to ECR, print the digest
```

Accept: `apply` and then `destroy` leave nothing behind. `infra/cloud.py audit` lists every resource tagged `project=synthrig` and must print nothing after `destroy`.

Step 0.3: GPU dev instance

- AMI: the latest **AWS Deep Learning Base GPU AMI (Ubuntu 22.04)**, looked up by SSM parameter or name filter, never a hard-coded AMI id. It ships with the NVIDIA driver, Docker and the NVIDIA container toolkit.
- The instance runs in a private subnet with the instance profile from 0.2. It has no key pair and no inbound security-group rules.
- Root EBS volume of at least 100 GB, gp3.
- Idle auto-stop: a CloudWatch alarm on low CPU **and** low GPU utilisation for `dev_idle_stop_minutes` triggers a stop action. (Publish GPU utilisation with the CloudWatch agent's `nvidia_gpu` metrics.)

CLI:

```
python infra/cloud.py dev up | down | status
python infra/cloud.py dev shell              # SSM session
python infra/cloud.py dev run -- <cmd>       # runs inside the container on the
dev box
python infra/cloud.py dev gpucheck           # see below
```

`gpucheck` (script at `infra/gpucheck.py`, run inside the container):

1. Print `nvidia-smi` output and Blender's Cycles device list.
2. Enable OptiX (fall back to CUDA).
3. Render the demo frame at scale 0.5 on GPU and then on CPU, and print both render times.
4. Assert that the GPU was actually used (the device list is non-empty and the GPU render is faster).

If option A (the pip `bpy` wheel) shows no GPU devices, switch the Dockerfile to option B and say so in the PR.

Accept: `dev up` gives a running box, `gpucheck` passes and reports OptiX, and `dev down` stops it. Report the GPU vs CPU render times for the demo frame in the PR.

Step 0.4: worker + S3 output (the shard contract)

`infra/worker.py` (the container entrypoint):

```
worker.py --config configs/runs/<x>.yaml --run-uri
s3://<bucket>/datasets/<scene>/<rig_hash>/<run_id>
          --shard-index ${AWS_BATCH_JOB_ARRAY_INDEX:-0} --num-shards N --
frames-total F
          [--scale s] [--samples n] [--device OPTIX|CUDA|CPU]
```

- **Frame range per shard:** contiguous blocks, `start = shard * ceil(F/N)`.
- For each frame in the range:
    1. Skip it if `_DONE` already exists in S3.

2. Otherwise render it to local scratch through the existing `Pipeline` (call `render_frame(idx)`; do not fork the pipeline).
3. Upload the frame dir, then write `_DONE`.
4. Delete the local frame dir.

- On SIGTERM (Spot interruption), finish or abandon the current frame cleanly and exit non-zero, so Batch retries. Resumability comes from `_DONE`.
- Stream logs to stdout (CloudWatch) and also upload `logs/shard_<k>.log` at the end.
- **Required change to `synthrig`** (small and additive): make the dataset header deterministic, and let it be written separately from rendering:
  - Add `Pipeline.write_header()`.
  - Add a `--header-only` path in `run.py`.
  - Move the wall-clock `created` field out of anything that shards compare.
  - Add a `generator_env` block to `manifest.json`: Blender version, image digest, git sha, render device, instance type (from IMDSv2, or `local`).

  `submit` writes the header once. Workers never write it.

Accept: locally, `worker.py` with 2 shards and a `file://` or MinIO-style local target (or a real bucket) produces the same frame set as a single `run.py` run. Killing a worker mid-shard and re-running it completes only the missing frames.

Step 0.5: AWS Batch (scale-out)

Terraform adds:

- A managed EC2 compute environment: GPU instance types from config, Spot with `SPOT_PRICE_CAPACITY_OPTIMIZED` when `use_spot`, plus an On-Demand environment as the second-priority fallback.
- min vCPU 0, max from config.
- The ECS GPU-optimised AMI (Batch default for GPU).
- A job queue.
- A job definition: the container image by digest, 1 GPU, memory/vCPU sized for `g6.xlarge`, the job role, and up to 3 retry attempts on Spot interruption (`evaluateOnExit` for host-level exit reasons).

CLI:

```
python infra/cloud.py submit --config configs/runs/app1_demo.yaml --frames 1000
--shards 50 [--scale ..] [--samples ..]
      # writes header to S3, submits array job, prints run URI + job id
python infra/cloud.py status <job-id|run-uri>   # shards done/failed, frames
with _DONE, ETA
python infra/cloud.py pull <run-uri> [--frames 0:10] --out out/<name>   #
download (subset) for local eval
python infra/cloud.py cost <run-uri>            # instance-hours from job
records; no hard-coded prices
```

Accept:

- A 4-shard × 2-frame job completes on Spot.
- `status` shows 8/8 frames.
- `pull` followed by `eval/preview.py` and `eval/dots_eval.py` works on the downloaded dataset.
- The compute environment returns to 0 instances afterwards.

Step 0.6: docs + guardrails

- `infra/README.md`: prerequisites (AWS CLI v2 with SSO, Docker, Terraform), first-time setup, the day-to-day commands, how to resume a failed run, and how to tear everything down.
- `cloud.py` refuses to run if `infra/config.yaml` still has placeholders, or if the git working tree is dirty during `submit`. A `--allow-dirty` flag overrides the dirty-tree check, and the manifest then records it.
- A CI job (GitHub Actions or equivalent) that builds the image and runs `image test` on every PR. Pushing to ECR from CI is optional and needs OIDC. Never store AWS keys in CI secrets.

Out of scope

- Kubernetes/EKS, SageMaker, or a web UI.
- Multi-region.
- Deleting existing datasets from `cloud.py`. Deletes are manual on purpose.